

Veckomöte v23

Deltagare: Eliot, Lene-Kristian

Check-In och sammanfattning plan framåt

- Nu lugnar saker ner sig! Jag lämnar opponeringen bakom mig: deltagandet + en skriftlig renskrivning av min feedback (inväntar att få den godkänd och rapporterad av Anders Ranzler på institutionen för reglerteknik) och opponeringens "muntliga del" blev godkänd som du (Lene-Kristian) mycket uppmärksammat lade märke till.
- Arbetet relaterad till exjobbet känner jag mig lugn och jag känner mig mer taggad. Jag har förtroende för att kunna komma vidare (och bli klar) under kommande perioden när du är på semester.
 - Jag är nöjd med mitt upplägg med 4 st modeller som utgör en bas för min studie + 1 "experimentell modell" där jag försöker göra en modell som liknar MLPF, fast för LDMX och där kan jag lägga så mycket tid jag känner för utan större krav.
- Tyvärr är det största träningsjobben jag skickat endast de som jag visade förra veckan. COSMOS hade planerat service window mån - ons denna veckan som jag ej kände till. Jag kan tyvärr inte ens logga in och kolla logs från det jag skickade iväg.
 - Jag skickade iväg 4 st slurm jobs med träning på 100 000 events + 1 000 000 events med de 2 arkitekturerna med ecal + tpad input.
 - Initiella resultat visar att training inference är tydligt långsammare med GNN gravnet, utan några uppenbara fördelar hittills.

Arbetet just nu

- Min kodbas på cosmos divergerade från de som jag har på mac / windows och jag har ordnat så att git fungerar som det ska på cosmos och jag kan koda på vilket ställe som helst och göra merge osv.
 - Jag kan skriva lite kod på mac, göra en pull på cosmos och köra den nya koden direkt.
- Denna veckan har jag suttit och förbättrat min kodbas ytterliggare och jag tror att det kan översättas till att min träning kommer att köra snabbare.
 - Träning med batches
 - Tidigare: single events i forward pass
 - Numera: batches av events i forward pass och sedan utnyttjas samtliga event i uppdatering av parametrar i träningen
 - Detta är standard sätt att parallelisera träningen ytterliggare och GPU resurser bör kunna utnyttjas mer effektivt, vi bör kunna se att träning tar mindre tid än förut. Detta återstår att se när cosmos öppnar igen ikväll/imorgon. **EDIT: Den har öppnat redan!**
 - Här är kanske en lösning på vår tidigare diskussion om att "jobba mer på bredden"
 - Transformer: lägger ihop en hel batch i en padded tensor och tränar på den
 - GNN: padding-logiken är inte applicerbar här. PyG-style batching "Concatenate node features/labels -> batch vector"
 - Hänger inte med riktigt vad som sker här återstår att testa lite.
 - Metrics logik är simplificerad
 - Använder torch.bincount och undviker for loops som traverserade över varje hit i ett event tidigare...
 - Det är mycket möjligt att en alldeles för stor del av tiden lades på detta i mina training loops...
- Implementerat en ny "vizualiser": en funktion som genererar en enda 3D plot event window med vanlig färgmarkering för korrekta prediktioner och ett rött kryss över inkorrekta prediktioner (med predikterade origin ID färgen förutom rött kryss).
 - Kan visa ett exempel på detta (obs ej tränad modell)
- Jag är lite orolig kring och har lite frågetecken som jag vet inte vet hur mycket jag kommer att lösa när det kommer till att försöka öka performance på mina 4 modeller som utgör basen för min rapport.
 - -> Jag har kikat mer på MLPF repo för att försöka dekonstruera och förstå hur deras arkitektur ser ut.
 - MLPF använder väldigt liknande transformer-liknande arkitektur med full self-attention
 - Modellen gör multi-task prediction i två kategorier ID classification + regression tasks: particle/no-particle, particle ID och kinematiska storheter (pn, sin(psi), cos())
 - Mest relevanta idéer att inspireras av:
 - separata heads för olika uppgifter,
 - event-/hit-context via transformer,
 - eventuellt separata representationsflöden för klassificering och regression,
 - element-type-aware encoding om olika inputs används, t.ex. ECal + trigger pad.
- Ifall jag reflekterar lite kort över vårterminen är det najs att fundera över platformen som har skapats:
 - Script som skapar en massa overlays och ett som transformerar dessa till pytorch shards redo för ML modeller
 - Ett ramverk för ML-baserade arbeten med input nodes/tokens från LDMX simuleringar. (Kodandet har definitivt accelererats med agentic AI också)
 - Gemensam bas root reading / IO / pytorch sharding
 - Gemensamma vizualisera
 - Flexibelt att byta mellan olika arkitekturer
 - Kan passera argument till träning + vald arkitektur
 - Kan passera argument som väljer input data osv
 - Gemensamma utils som ser till att
 - Git repo med stöd för att köra koden på windows/mac/linux system.
 - Get repo med stöd för att ladda in LDMX-SW med docker i VS Code på Mac/Linux system (och därmed aktivera intellisense för LDMX-SW koden!)
 - **Allt som allt har det tagit tid att bygga upp allt detta, men nu är allting mer eller mindre på plats och det återstår fortfarande att se hur långt jag kan komma med träning. Oavsett vad planerar jag att lägga utrymme för att förklara kodbasen i rapporten och evaluera/presentera ML ramverket som ett resultat i sig.**

Datum inlämning/presentation

- Spikat!
 - Preliminär deadline för mig själv: 1 augusti
 - Inlämning Oxana: 7 augusti
 - Presentation: 21 augusti kl 14:00

Anteckningar från mötet